*Load-Aware A Path Planning for Quadruped Robots: Daily Progress Report**

Timeline: Day 2 17.4.26 (Week 1: Setup & Initial Implementation)

NAME : HARINI P (Department of Computer science and Engineering)

1. Nature of Work

Day 1 focused on environment setup and foundational coding for the grid-based simulation environment used by the Load-Aware A* algorithm. The primary objective was to establish a clean, reproducible development environment in Google Colab and implement the Environment class that will serve as the simulation grid for all path planning experiments. This included designing the grid data structure, implementing obstacle placement, rendering the grid visually, and documenting the code structure for team sharing.

2. Tasks Completed

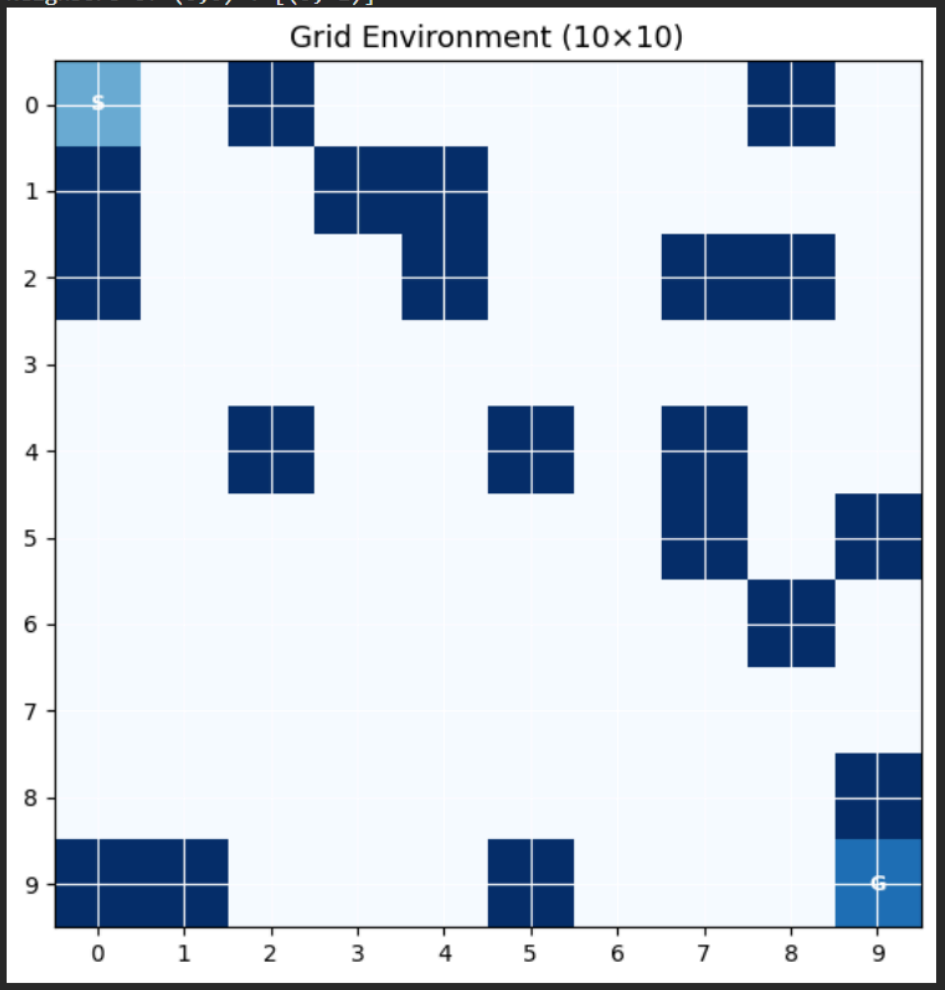
- **Setup Google Colab**
  Opened a new notebook, set runtime to Python 3, installed NumPy and Matplotlib. Verified all library imports execute without errors.
- **Create Environment class**
  Implemented the core grid class with configurable grid size, start/goal positions, and random obstacle generation method.
- **Test basic grid creation**
  Successfully instantiated a 10×10 grid, placed obstacles, and verified start/goal positions. Rendered the grid using Matplotlib to confirm visual output.
- **Document code structure**
  Added inline comments to all class methods and created a short README block at the top of the notebook explaining class attributes, method signatures, and usage example.

3. Proof of Work — Code

```python
import numpy as np
import matplotlib.pyplot as plt
import random
class Environment:
    """
    Grid-based 2D simulation environment for quadruped path planning.
    Attributes:
        rows  (int)  : Number of grid rows
        cols  (int)  : Number of grid columns
        grid  (ndarray): 2D numpy array (0=free, 1=obstacle)
        start (tuple): Start cell (row, col)
        goal  (tuple): Goal cell (row, col)
    """
    def __init__(self, rows=10, cols=10,
                 start=(0,0), goal=(9,9),
                 obstacle_prob=0.2, seed=42):
        """Initialize the grid environment."""
        self.rows   = rows
        self.cols   = cols
        self.start  = start
        self.goal   = goal
        random.seed(seed)
        np.random.seed(seed)
        # Build grid: 0 = free cell, 1 = obstacle
        self.grid = np.zeros((rows, cols), dtype=int)
        self._place_obstacles(obstacle_prob)
    def _place_obstacles(self, prob):
        """Randomly place obstacles; start and goal are always free."""
        for r in range(self.rows):
            for c in range(self.cols):
                if (r, c) not in (self.start, self.goal):
                    self.grid[r][c] = 1 if random.random() < prob else 0
    def is_free(self, row, col):
        """Return True if cell is within bounds and not an obstacle."""
        return (
            0 <= row < self.rows and
            0 <= col < self.cols and
            self.grid[row][col] == 0
        )
    def get_neighbors(self, row, col):
        """Return list of free 4-connected neighbor cells."""
        directions = [(-1,0),(1,0),(0,-1),(0,1)]
        return [(row+dr, col+dc)
                for dr, dc in directions
                if self.is_free(row+dr, col+dc)]
    def random(self):
```

```python
    def render(self):
        """Visualize the grid using matplotlib."""
        display = self.grid.copy().astype(float)
        display[self.start] = 0.5   # start cell
        display[self.goal]  = 0.75  # goal cell
        fig, ax = plt.subplots(figsize=(6, 6))
        ax.imshow(display, cmap='Blues', vmin=0, vmax=1)
        ax.set_title('Grid Environment (10×10)', fontsize=13)
        ax.set_xticks(range(self.cols))
        ax.set_yticks(range(self.rows))
        ax.grid(True, color='white', linewidth=0.8)
        # Annotate start and goal
        ax.text(self.start[1], self.start[0], 'S',
                ha='center', va='center', fontsize=9,
                fontweight='bold', color='white')
        ax.text(self.goal[1],  self.goal[0],  'G',
                ha='center', va='center', fontsize=9,
                fontweight='bold', color='white')
        plt.tight_layout()
        plt.show()
# ── Usage Example ──────────────────────────────────────────
env = Environment(rows=10, cols=10,
                  start=(0,0), goal=(9,9),
                  obstacle_prob=0.2, seed=42)
print(f"Grid shape : {env.grid.shape}")
print(f"Start cell : {env.start}")
print(f"Goal cell  : {env.goal}")
print(f"Obstacles  : {int(env.grid.sum())} cells blocked")
print(f"Neighbors of (0,0) : {env.get_neighbors(0,0)}")
env.render()
```

```
Grid shape : (10, 10)
Start cell : (0, 0)
Goal cell  : (9, 9)
Obstacles  : 19 cells blocked
Neighbors of (0,0) : [(0, 1)]
```



4. Code Structure Documentation

The notebook is organized into 3 cells:

1. **Cell 1 — Imports & Setup**

   Imports NumPy, Matplotlib, and random. Sets global random seed for reproducibility.

2. **Cell 2 — Environment Class Definition**

   Full class with __init__, _place_obstacles, is_free, get_neighbors, and render methods. All methods are fully commented with docstrings.

3. **Cell 3 — Usage Demo**

   Instantiates a 10×10 grid, prints metadata, tests is_free and get_neighbors, and calls render() to produce the visualization.

5. Observations & Notes

The Environment class is designed to be directly reusable by Members 2 and 3 for Dijkstra, RRT, and testing setups respectively — all algorithms will share the same grid structure. Obstacle density is parameterized via obstacle_prob to allow easy scaling for Scenarios 1–20 in later days. The seed parameter ensures identical grids are reproducible across all team members.